School of Computing
CISC 886 - Cloud Computing

Cloud-Based Math Q&A Chatbot

Final Project Report

End-to-End AWS Workflow with S3, EMR, QLoRA Fine-Tuning, Ollama, and OpenWebUI

Pipeline Summary

StackMathQA → Amazon S3 → AWS EMR + PySpark → LLaMA 3 8B QLoRA fine-tuning → GGUF artifact → EC2 + Ollama → OpenWebUI browser assistant

Team Members

Name	Student ID
Ahmed Mohamed Al Shobaki	20596291
Fatma Yousuf Abu Al Wafa	20596315
Zainb Maged Arafa Zahran	20596293

Course	CISC 886 - Cloud Computing
AWS Region	`us-east-1` / US East (N. Virginia)
Resource Prefix	`25qgkp`
GitHub Repository	`https://github.com/zainbmaged/Cloud_project_Group6.git`

Contents

Executive Summary

This report documents the complete implementation of a cloud-based mathematical question-answering assistant for CISC 886. The final system is not only a model demo; it is an end-to-end cloud workflow that starts with raw StackMathQA data, preprocesses it with Apache Spark on AWS EMR, fine-tunes a LLaMA 3 8B model using QLoRA, exports the model as a quantized GGUF artifact, and serves the model from an AWS EC2 instance through Ollama and OpenWebUI.

The project work was divided into three practical areas: infrastructure and deployment, data preprocessing, and model fine-tuning. The infrastructure work created the custom AWS networking environment, EC2 deployment, Ollama serving layer, OpenWebUI interface, API tests, screenshots, and cost evidence. The data work built the S3 layout, EMR + PySpark preprocessing pipeline, quality filtering, EDA, and train/validation/test outputs. The model work selected LLaMA 3 8B, fine-tuned it with Unsloth + QLoRA, evaluated it, and exported the deployable GGUF artifact. The final integration connected these pieces into a working browser-based chatbot.

The report intentionally explains both **what was configured** and **why each decision was made**, because the project instructions emphasize justification of the VPC design, preprocessing approach, fine-tuning method, deployment commands, web interface evidence, and cost/cleanup evidence.

1 System Architecture

1.1 Architecture Overview

The architecture is organized into three zones to make the system easy to understand and to separate responsibilities. Zone 1 is the data and model preparation pipeline. Zone 2 is the AWS deployment environment inside the custom VPC. Zone 3 is the user access layer, where the browser and terminal interact with the deployed system.

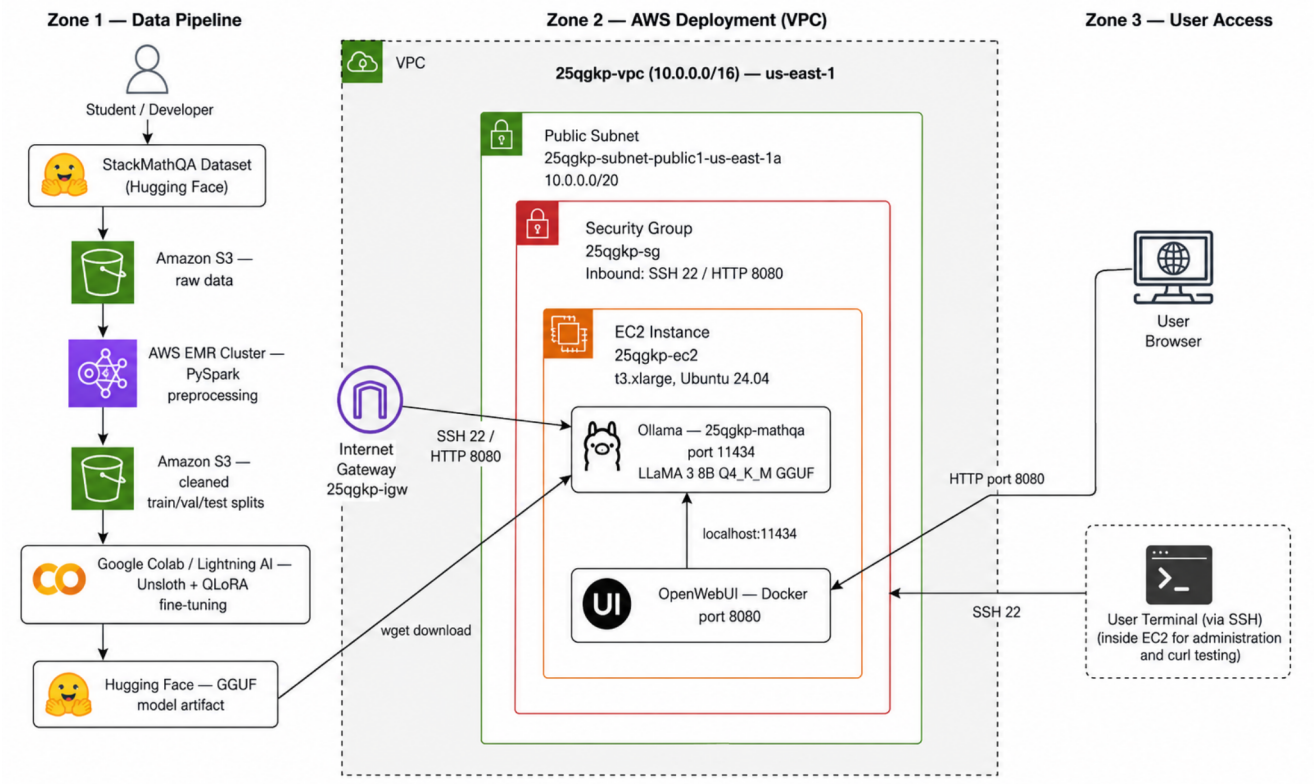

Figure 1: Final three-zone system architecture used for the project. The diagram connects the StackMathQA data pipeline, AWS VPC deployment, Ollama model runner on localhost port 11434, OpenWebUI on port 8080, and user access paths.

1.2 End-to-End Data and Request Flow

The data flow begins with the StackMathQA dataset from Hugging Face. The raw dataset is exported as JSONL and uploaded to Amazon S3. AWS EMR then runs a PySpark preprocessing job that reads the raw JSONL file, cleans and filters the question-answer records, computes EDA outputs, deduplicates records, formats them into an instruction-following schema, and writes clean outputs back to S3. These cleaned outputs are used by the fine-tuning notebook in Lightning AI / Google Colab.

After fine-tuning, the model is exported as a GGUF file because Ollama can load GGUF models efficiently on a CPU-only EC2 instance. The EC2 instance is inside the public subnet of the custom VPC. Ollama listens locally on port `11434`, and OpenWebUI connects to Ollama through `localhost:11434`. Users do not need direct access to Ollama. They use a browser to access OpenWebUI on port `8080`, while administrative access is performed over SSH on port `22`.

This design keeps the data pipeline and deployment pipeline logically separate. EMR is used only for preprocessing and then terminated. EC2 is used only for serving the model and web interface. This separation reduces cost and makes the final system easier to explain, reproduce, and grade.

1.3 Why This Architecture Fits the Project

The project requires a complete cloud-based assistant, not only local fine-tuning. This architecture satisfies that requirement because it includes cloud storage, distributed preprocessing, model adaptation, model serving, and a browser-based interface. The architecture also reflects the real constraints of the shared AWS account. Instead of using an expensive GPU instance for inference, we used a quantized 4-bit GGUF model and deployed it on a `t3.xlarge` CPU instance. This is slower than GPU inference, but it is practical for a short course demo and keeps the deployment within reasonable resource limits.

2 VPC and Networking

2.1 Console Provisioning Choice

The VPC and networking resources were created using the AWS Console rather than Terraform. This was an acceptable choice for this project because the deliverable allowed either Terraform files or annotated console screenshots with written justification. Since the deployment was temporary and the grader needed visible proof of the configuration, the Console approach made it easier to capture direct evidence of the VPC, subnet, route table, Internet Gateway, security group, and EC2 settings.

This does not mean Terraform is less valuable. In a production or repeated deployment, Terraform would be the better option because it provides version-controlled infrastructure. For this course project, the Console approach was chosen because it reduced setup overhead, made screenshots straightforward, and still produced a reproducible configuration documented in the README.

2.2 VPC and Subnet Design

A new custom VPC named `25qgkp-vpc` was created with CIDR block `10.0.0.0/16`. The default AWS VPC was not used. The `/16` CIDR provides a large private address space, even though the current project uses only one EC2 instance. This leaves room for future expansion, such as adding private subnets, a NAT Gateway, a bastion host, or additional services.

The public subnet was named `25qgkp-subnet-public1-us-east-1a` and assigned CIDR block `10.0.0.0/20`. A single public subnet was enough because the final deployment required a public OpenWebUI demo, not a high-availability multi-AZ production system. A private subnet was not necessary for the final demo, and avoiding NAT Gateway also avoided extra cost.

Resource	Name / ID	Reason
VPC	`25qgkp-vpc` / `vpc-024f5f93ab95a33f1`	Isolates the project from the default VPC and satisfies the custom VPC requirement.
VPC CIDR	`10.0.0.0/16`	Provides enough private IP space for future expansion.
Public subnet	`25qgkp-subnet-public1-us-east-1a` / `subnet-0831c724796053ea0`	Hosts the EC2 instance that must be reachable for SSH and OpenWebUI.
Subnet CIDR	`10.0.0.0/20`	Gives thousands of usable private IPs, more than enough for this temporary deployment.

Internet Gateway	`25qgkp-igw` / `igw-04f7ea4b18fc3062f`	Allows internet access for SSH, package installation, Hugging Face download, and browser access.
Route table	`25qgkp-rtb-public`	Sends public traffic to the Internet Gateway.
Security group	`25qgkp-sg` / `sg-07fc2d4b20b112492`	Controls inbound access to the EC2 deployment instance.

2.3 Internet Gateway and Route Table

The Internet Gateway `25qgkp-igw` was attached to the VPC. The public route table `25qgkp-rtb-public` included the local route `10.0.0.0/16 -> local` and a default internet route `0.0.0.0/0 -> 25qgkp-igw`. The local route allows communication within the VPC, and the default route allows the EC2 instance to access the internet and receive inbound traffic on the allowed ports.

2.4 Security Group Design

The final security group exposed only the ports needed for the demo. SSH port 22 was opened for administration, while TCP port 8080 was opened for OpenWebUI browser access. Ollama's port 11434 was kept local to the EC2 instance and was not exposed publicly in the final security group. This is an important design choice: OpenWebUI talks to Ollama through `localhost:11434`, so users do not need direct external access to the model API.

Type	Port	Source	Justification
SSH	22	`0.0.0.0/0`	Used temporarily for EC2 setup, model loading, Docker installation, and testing. In production this should be restricted to the administrator's IP address.
Custom TCP	8080	`0.0.0.0/0`	Allows the grader/user to access OpenWebUI from a browser.
Ollama API	11434	Localhost only	Not publicly exposed in the final security group. OpenWebUI connects internally through `localhost:11434`.

2.5 Networking Evidence

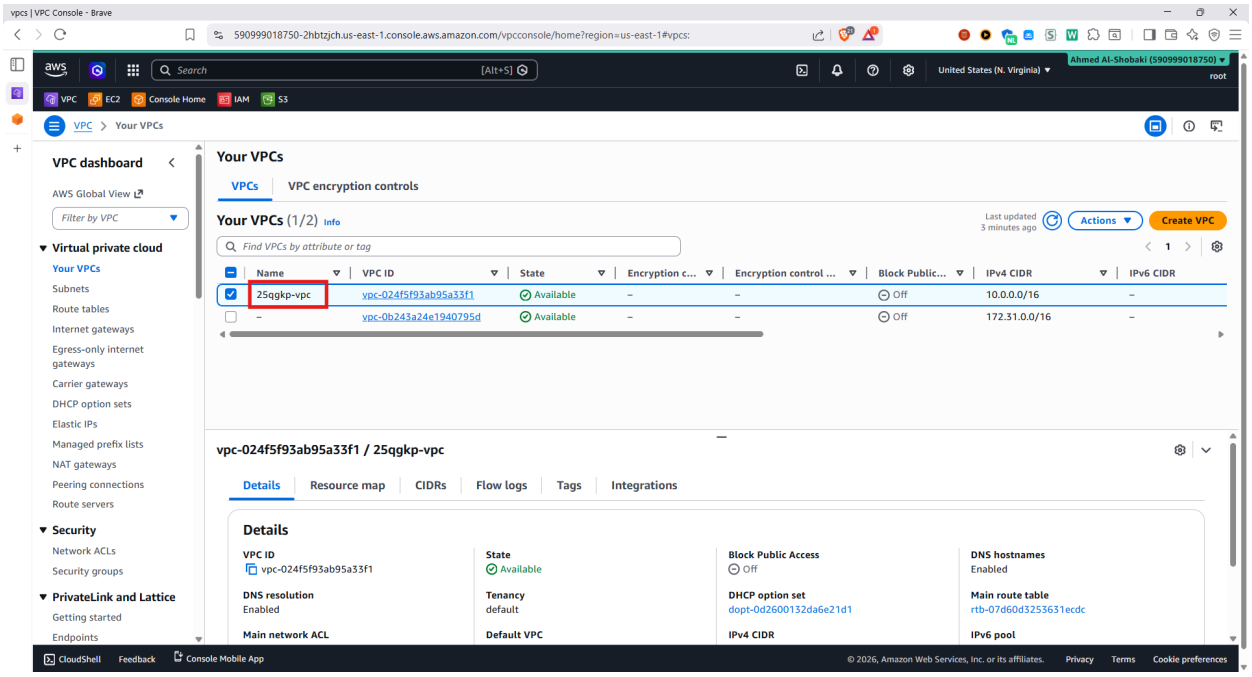

Figure 2: Custom VPC `25qgkp-vpc` with CIDR block `10.0.0.0/16`.

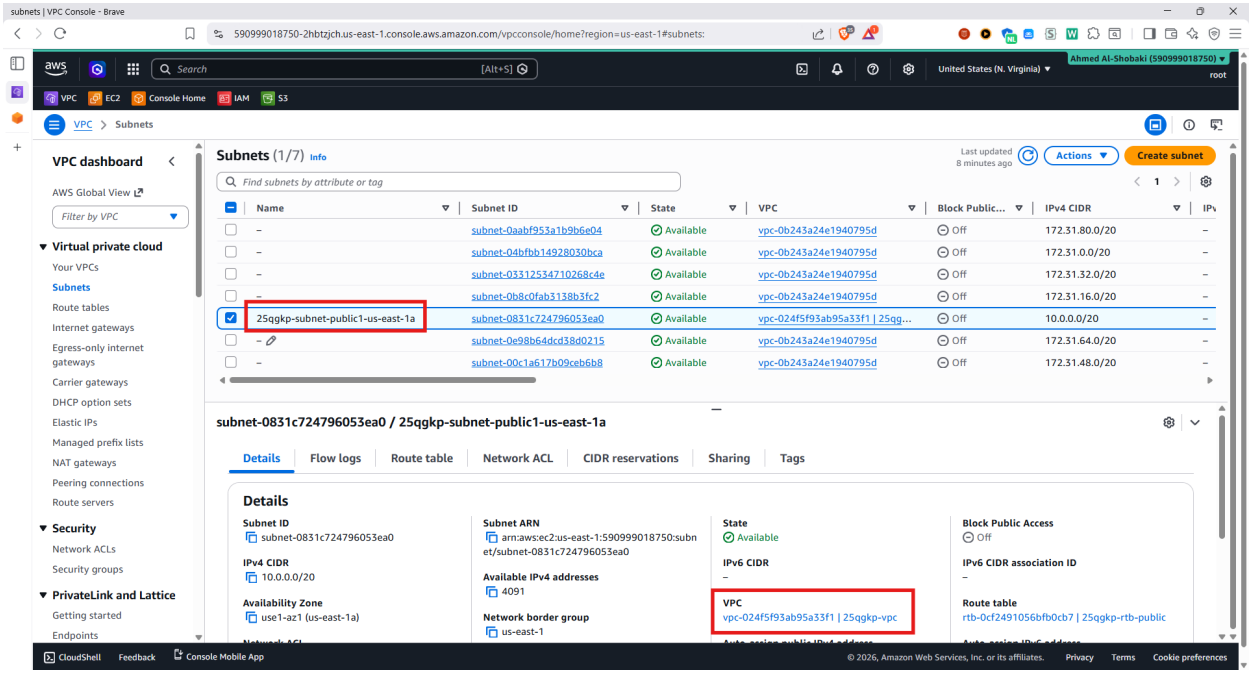

Figure 3: Public subnet `25qgkp-subnet-public1-us-east-1a` associated with the custom VPC.

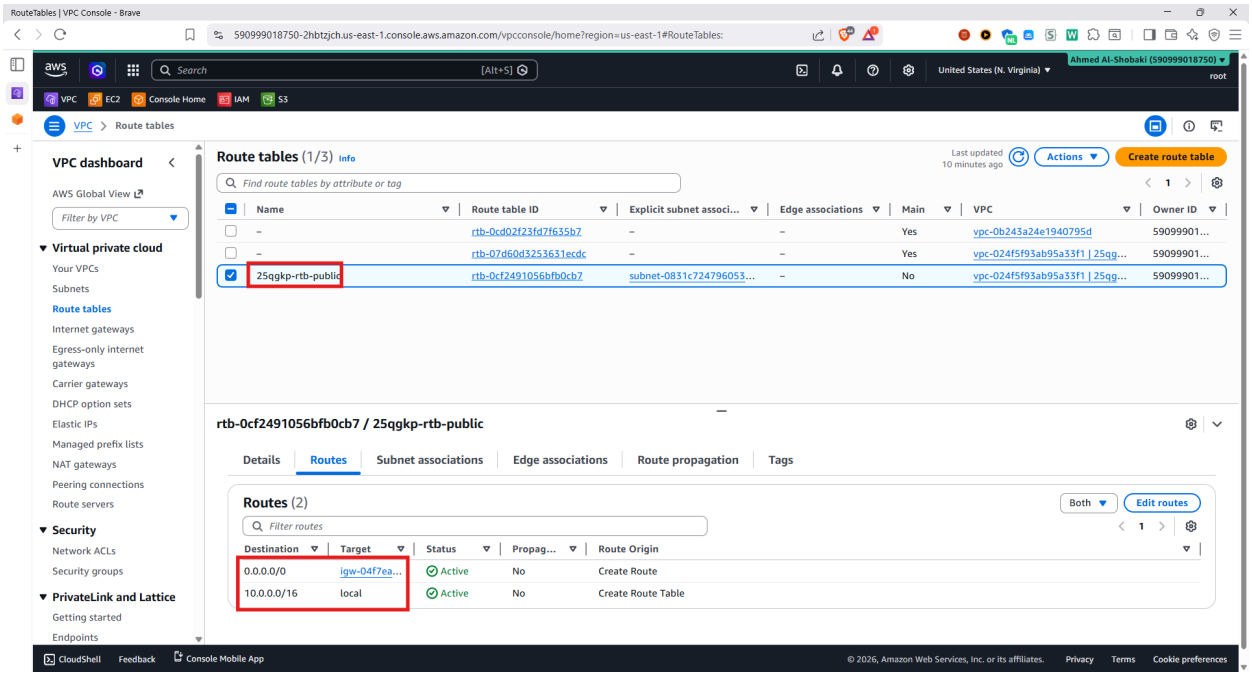

Figure 4: Public route table showing the local VPC route and the default route to the Internet Gateway.

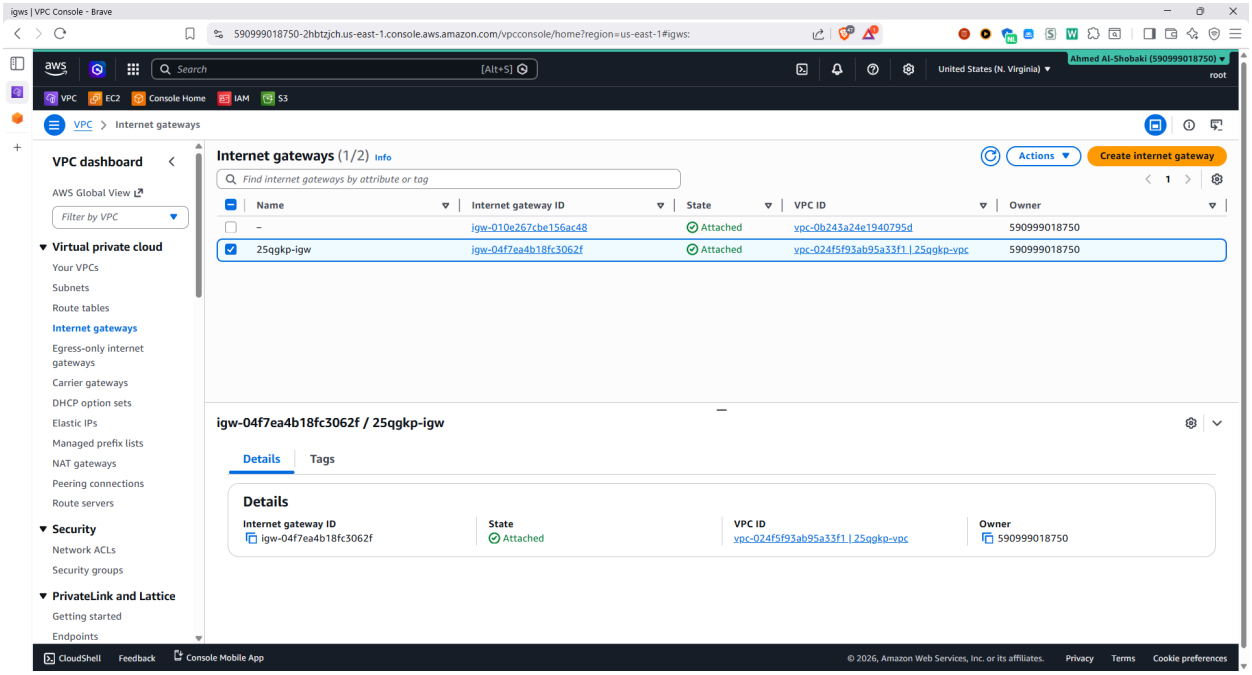

Figure 5: Internet Gateway `25qgkp-igw` attached to the custom VPC.

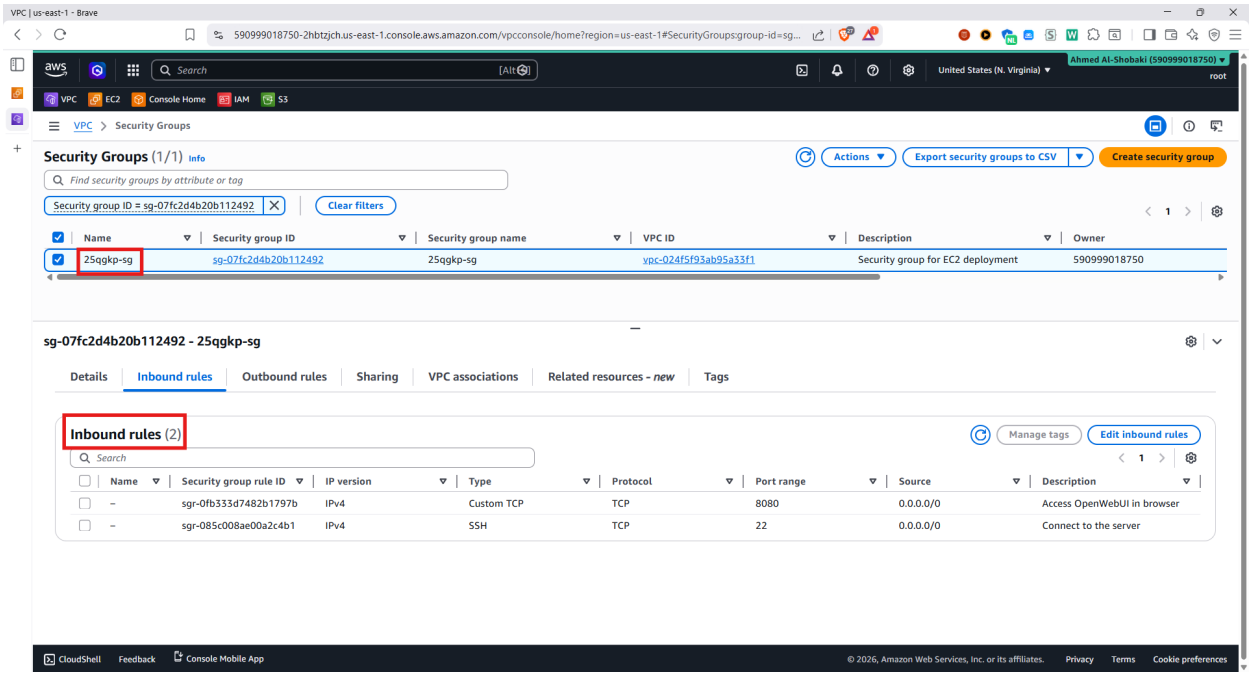

Figure 6: Security group `25qgkp-sg` with inbound rules for SSH and OpenWebUI.

3 Model and Dataset Selection

3.1 Dataset Selection: StackMathQA

The dataset selected for the project was StackMathQA, specifically the large 1600K subset from Hugging Face. This dataset was appropriate because it contains natural-language math questions paired with accepted or high-quality answers from Stack Exchange communities. The answers often include LaTeX notation and multi-step mathematical reasoning, which aligns well with the goal of building a Math Q&A chatbot.

Item	Value
Dataset name	StackMathQA - 1600K subset
Source	`https://huggingface.co/datasets/math-ai/StackMathQA`
License	CC BY-SA 4.0, derived from Stack Exchange content
Raw records used in preprocessing	1,600,000 records
Cleaned records after EMR preprocessing	360,032 records
Clean split	287,928 train / 36,052 validation / 36,052 test
Fine-tuning subset	50,000 samples loaded for model training, with 200 held out for evaluation
Domain coverage	Algebra, calculus, proof-based math, probability, linear algebra, statistics, and related math topics

3.2 Why StackMathQA Fits the Task

The project goal was not to build a generic chatbot. The objective was to adapt a model toward mathematical explanation. StackMathQA is suitable because it contains examples of how people ask

mathematical questions and how experienced users answer them. This includes formulas, proof-style reasoning, derivative and algebra questions, and explanatory text around equations. The dataset also naturally contains LaTeX, so the fine-tuned model learns to produce math notation that OpenWebUI can render cleanly.

The main risk with Stack Exchange style data is uneven quality. Some posts are too short, incomplete, duplicated, or contain irrelevant discussion. That is why preprocessing was necessary before fine-tuning. The EMR pipeline applied length filters, score-based filtering, cleaning, deduplication, and Alpaca-style formatting before the data was used by the model team.

3.3 Representative Cleaned Sample

The following example shows the type of instruction-output pattern used after preprocessing. It demonstrates why this dataset is useful for an assistant that must explain mathematical reasoning rather than only output a final number.

```
Instruction:
Can anyone explain to me how to use induction on this problem:
Prove by induction that for all n in N,
    sum_{k=0}^n 2k = n(n+1)

Response:
For n = 0, the left side is 0 and the right side is 0(1) = 0, so the base case holds.
Assume the formula is true for n = m, so sum_{k=0}^m 2k = m(m+1).
For n = m + 1, add 2(m+1) to both sides:
    sum_{k=0}^{m+1} 2k = m(m+1) + 2(m+1)
                       = (m+1)(m+2).
Therefore the statement holds by induction.
```

3.4 Model Selection: LLaMA 3 8B

The base model selected for fine-tuning was `unsloth/llama-3-8b-bnb-4bit`. It has 8 billion parameters and was used through Unsloth's 4-bit quantized version. This choice balanced model quality with the available hardware. Larger models might produce better answers, but they would be harder to fine-tune and deploy under the project constraints. Smaller models might run faster but would likely have weaker mathematical reasoning.

Item	Value
Base model	`unsloth/llama-3-8b-bnb-4bit`
Parameters	8 billion
Trainable parameters after LoRA	83,886,080, approximately 1.03% of total parameters
License	Meta LLaMA 3 Community License
Fine-tuning method	LoRA + QLoRA
Training hardware	Single NVIDIA L4 GPU, approximately 20 GB VRAM
Deployment format	GGUF, Q4_K_M quantization
Final deployed model	`25qgkp-mathqa-merged4:latest`
GGUF source	`https://huggingface.co/Zainb114/llama3-math-merged4-Q4_K_M-GGUF`

3.5 Hardware Fit and Deployment Fit

The 4-bit QLoRA version of LLaMA 3 8B was practical for training because the frozen model weights require much less VRAM than full precision weights. LoRA then trains only small adapter matrices instead of updating all 8 billion parameters. This allowed training on a single NVIDIA L4 GPU.

For deployment, the final GGUF file was around 4.9 GB. This made it suitable for Ollama on a `t3.xlarge` EC2 instance with 16 GB RAM. CPU inference is not fast, but it was acceptable for a demonstration because the goal was to show a complete cloud deployment rather than optimize latency.

4 Data Preprocessing with Apache Spark on EMR

4.1 Purpose of the EMR Stage

The preprocessing stage was required because raw StackMathQA data is too noisy to send directly into fine-tuning. The dataset contains many useful records, but also short answers, duplicated content, low-scored questions, and formatting inconsistencies. AWS EMR was used to run the PySpark job because it provides managed Spark infrastructure and supports larger-scale preprocessing than a local notebook.

The preprocessing pipeline follows this path:

```
S3 raw/all.jsonl -> EMR PySpark job -> S3 processed/clean_data + splits + EDA
```

4.2 EMR Cluster Configuration

The cluster was created in `us-east-1`. It used one master node and two core nodes, all `m5.xlarge`. This was enough for the dataset size while staying reasonable in the shared course account. A bootstrap script installed the Python libraries needed by the PySpark job and EDA code.

Parameter	Value
Cluster name	`25qgkp-emr`
Region	`us-east-1`
EMR release	`emr-7.1.0`
Applications	Spark
Master node	1 x `m5.xlarge`
Core nodes	2 x `m5.xlarge`
Task nodes	0
Log path	`s3://25qgkp-all-data/logs/`
Bootstrap script	`s3://25qgkp-all-data/scripts/bootstrap.sh`
PySpark script	`s3://25qgkp-all-data/scripts/25qgkp_emr_preprocessing.py`

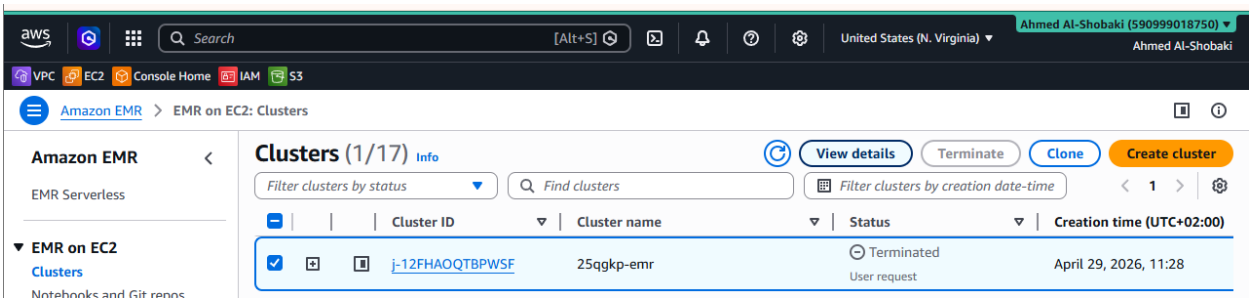

Figure 7: EMR console showing cluster `25qgkp-emr` during preprocessing.

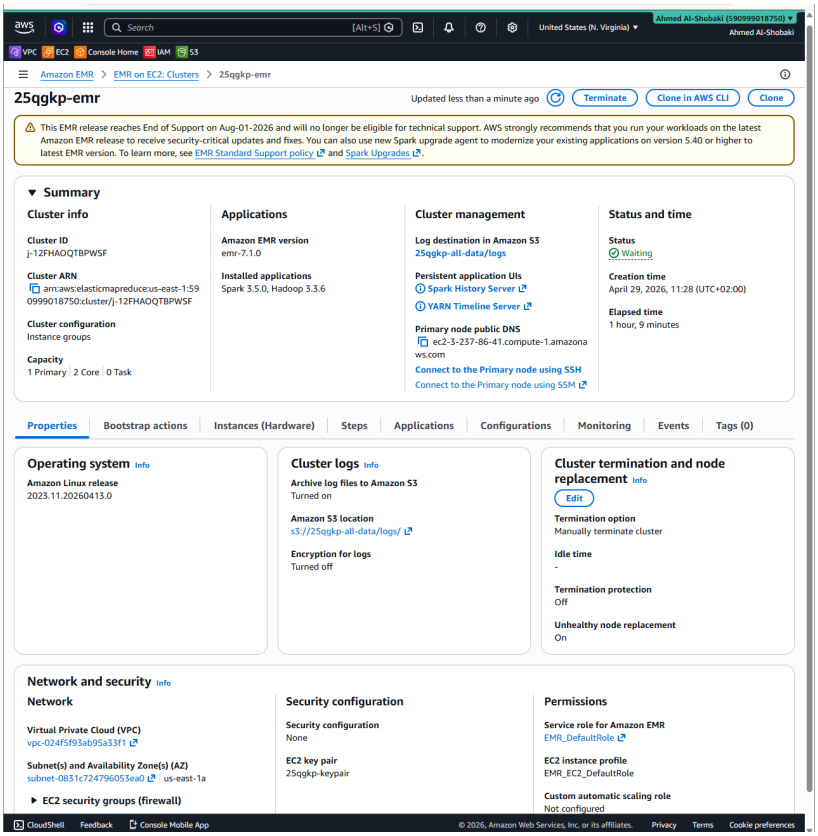

Figure 8: EMR instance groups showing one primary node and two core nodes.

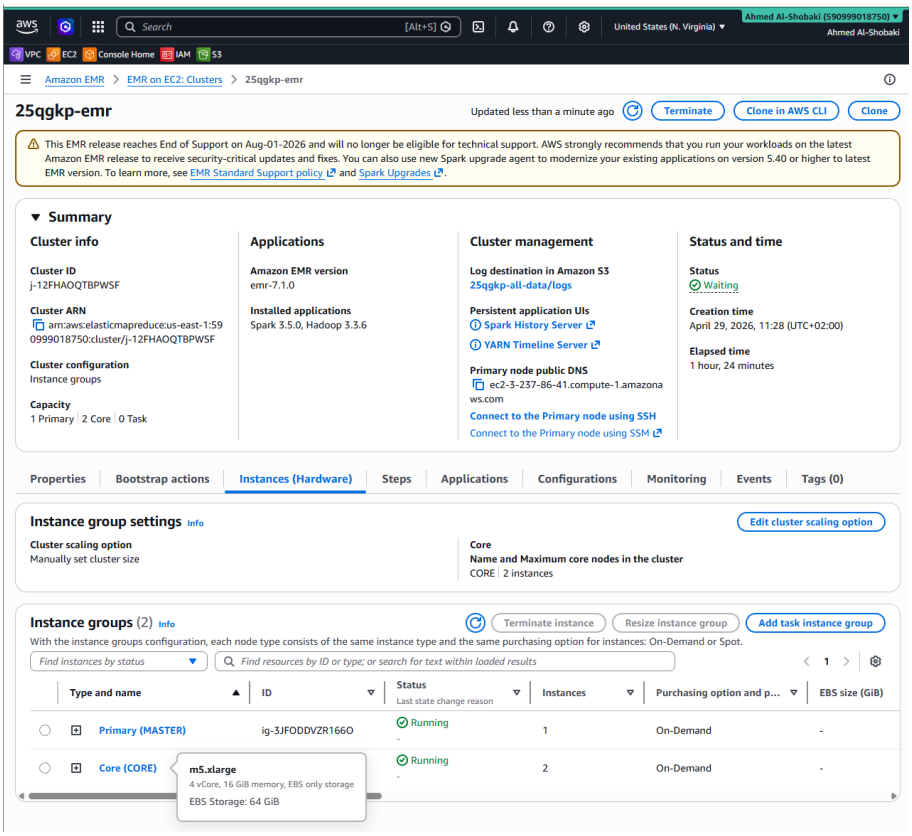

Figure 9: EMR hardware and cluster details, including the `m5.xlarge` configuration.

4.3 Bootstrap and Dependencies

The bootstrap step installed libraries used for reading data, writing to S3, and generating EDA figures. The important point is that the Python environment on the EMR nodes had to match the PySpark script dependencies before the job started.

```
#!/bin/bash
sudo pip3 install --upgrade pip
sudo pip3 install matplotlib numpy pandas pyarrow fsspec s3fs boto3
sudo pip3 install datasets --ignore-installed --no-deps
sudo pip3 install huggingface-hub tqdm requests filelock --ignore-installed
```

4.4 PySpark Preprocessing Logic

The PySpark script was committed to GitHub and also uploaded to S3. The code was organized as a reproducible pipeline rather than a manual notebook-only process. The main steps were:

1. **Read raw JSONL from S3:** The job loaded `s3://25qgkp-all-data/raw/all.jsonl` as the input dataset.
2. **Remove empty records:** Rows with missing or empty question/answer fields were removed because they cannot teach the model useful behavior.
3. **Normalize text and LaTeX:** Text cleaning reduced formatting noise while preserving mathematical notation.
4. **Apply length filters:** Very short questions/answers were likely uninformative, while extremely long samples could exceed training context limits.
5. **Apply quality filtering:** The pipeline kept questions with score at least 5. This threshold helped keep higher-quality Stack Exchange examples.
6. **Deduplicate before splitting:** Duplicate URL/answer ID pairs and content hashes were removed before train/validation/test splitting. This reduced leakage risk.
7. **Format as instruction-following data:** Cleaned records were converted into Alpaca-style fields: `instruction`, `input`, and `output`.
8. **Generate EDA:** Summary plots were created to inspect text lengths, quality distribution, and math/LaTeX readiness.
9. **Split and write outputs:** The clean dataset was split using an 80/10/10 ratio with a fixed random seed and written back to S3.

4.5 Preprocessing Configuration and Output Counts

Setting	Value
Minimum question length	40 characters
Maximum question length	1,500 characters
Minimum answer length	100 characters
Maximum answer length	4,000 characters
Minimum question score	5
Split method	PySpark `randomSplit`
Split ratio	80% train / 10% validation / 10% test

Random seed	42

Stage	Rows
Raw dataset	1,600,000
Final clean dataset	360,032
Train split	287,928
Validation split	36,052
Test split	36,052

The retention rate was approximately 360,032/1,600,000 = 22.5%. This is reasonable because the pipeline intentionally removed low-quality, duplicated, too-short, or too-long records. The final split counts sum back to the clean dataset count, which verifies that no rows were lost after the split stage.

4.6 S3 Evidence and EDA

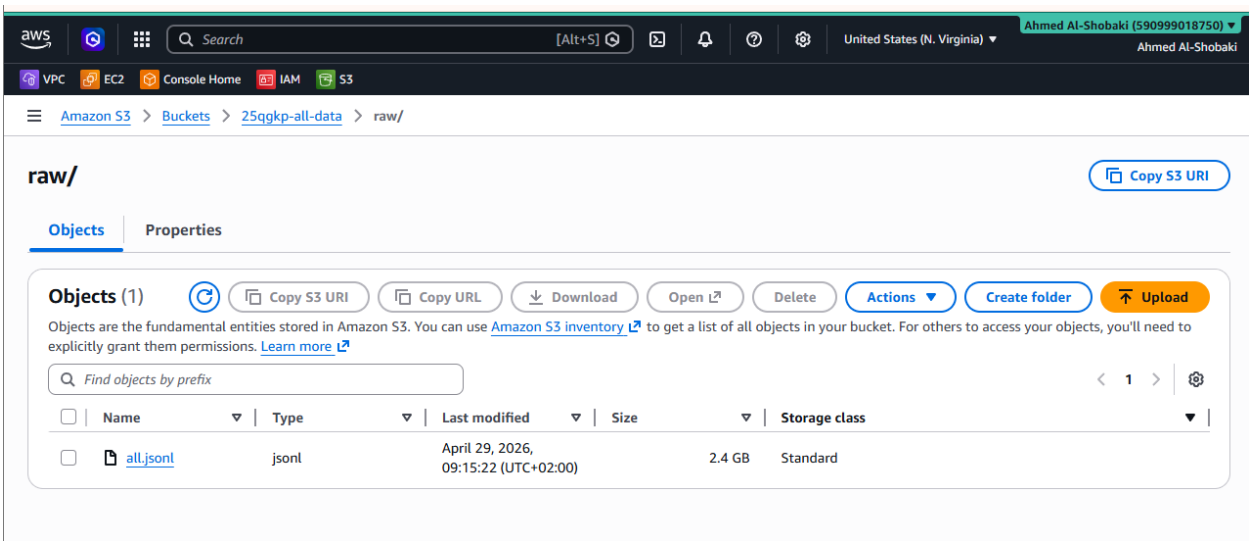

Figure 10: S3 raw folder containing `all.jsonl`, the raw StackMathQA input file for EMR.

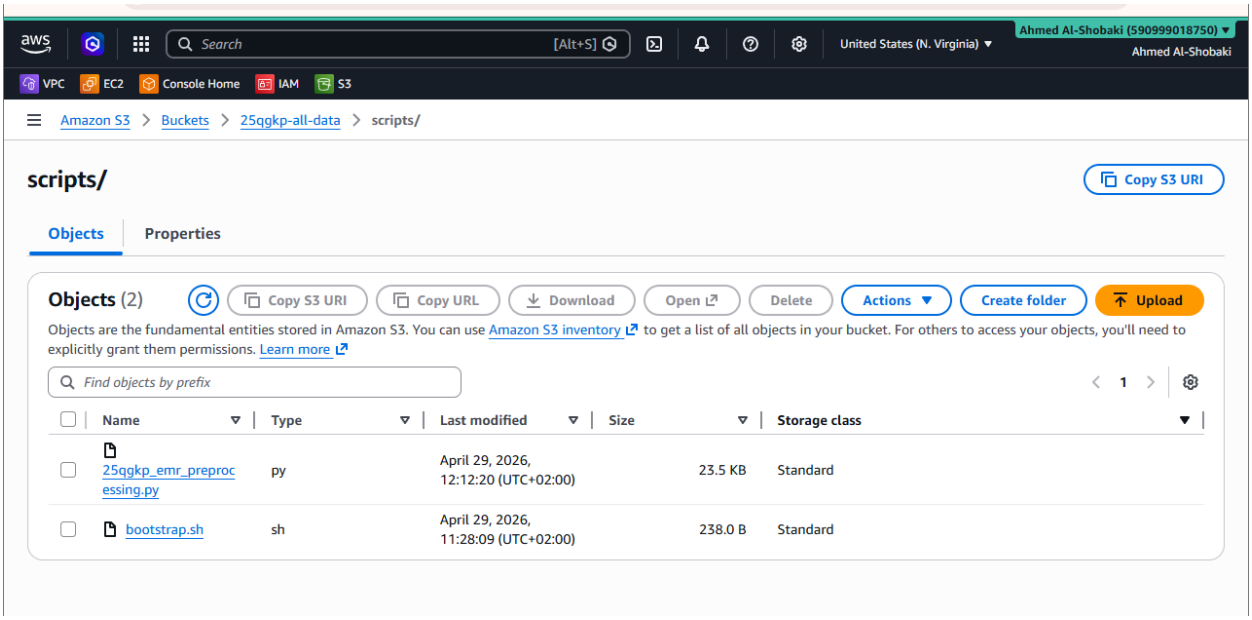

Figure 11: S3 scripts folder containing the PySpark preprocessing script and bootstrap script.

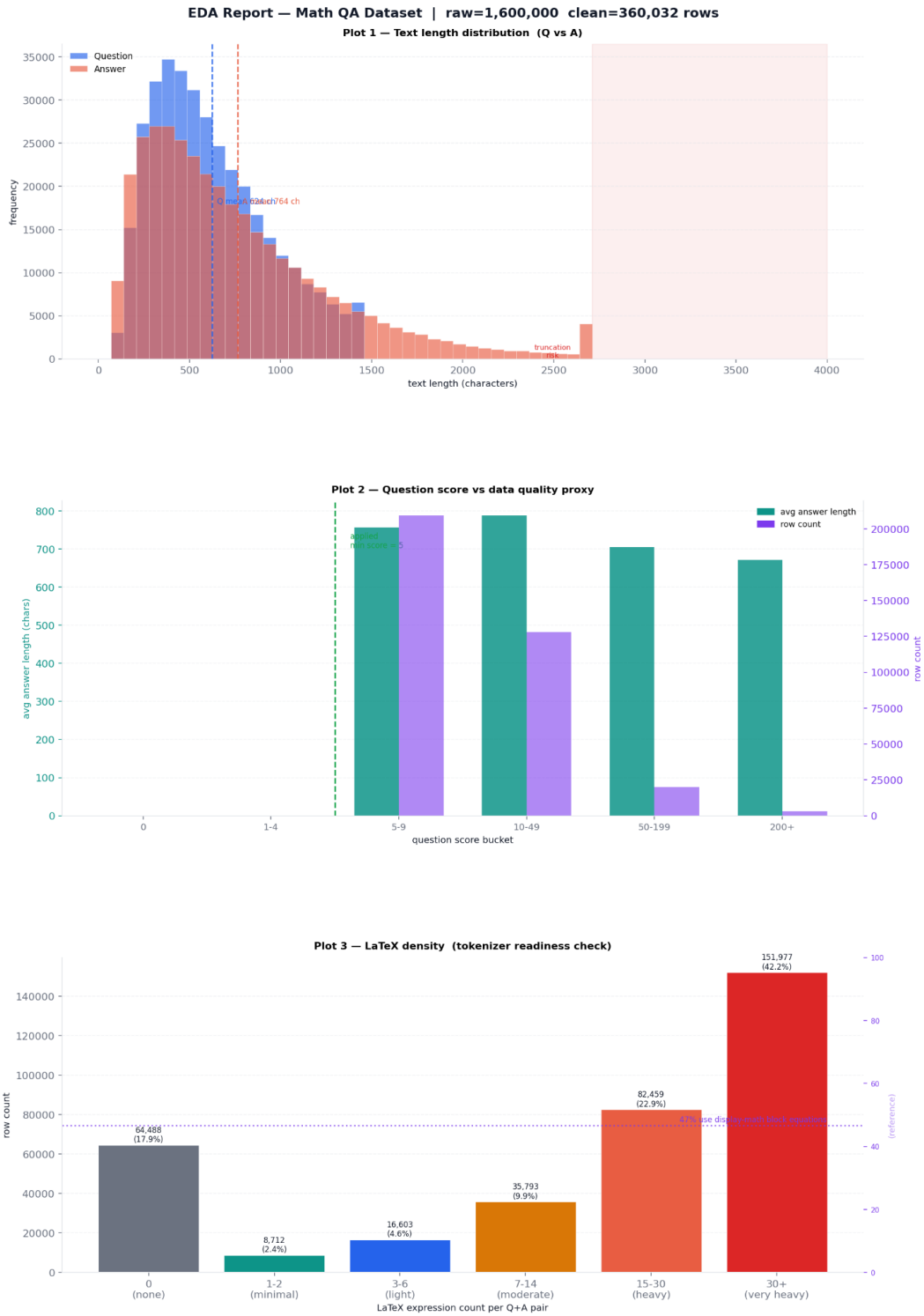

Figure 12: Combined EDA output. The three subfigures show text length distribution, question score/data-quality proxy, and LaTeX density/tokenizer readiness.

The EDA plots were used as quality checks rather than decorative figures. Each plot was used to verify a different part of the preprocessing decision.

Plot 1 - Text Length Distribution (Q vs A): This histogram compares the character length distribution of questions and answers across the 360,032 clean records. Question lengths are concentrated between 200 and 700 characters (mean: 624, median: 563), while answer lengths are more spread out and longer on average (mean: 764, median: 624, max: 4,000). A truncation-risk zone is marked at 4,000 characters to flag records that may be cut off during tokenization.

Plot 2 - Question Score vs Data Quality Proxy: This dual-axis bar chart groups records by

question-score bucket (5–9, 10–49, 50–199, 200+) and plots both the row count and the average answer length per bucket. Higher-scored questions tend to attract longer and more detailed answers, confirming that the minimum score threshold of 5 is a meaningful quality filter. The chart also shows that the majority of retained records fall in the 5–9 score bucket, with progressively fewer but higher-quality records in higher buckets.

Plot 3 - LaTeX Density / Tokenizer Readiness Check: This bar chart shows the distribution of LaTeX expression counts per question-answer pair. Approximately 17.9% of records contain no LaTeX, while 42.2% contain 30 or more LaTeX expressions and are therefore classified as very heavy. This confirms that the dataset is strongly mathematical and that the chosen model and tokenizer must support LaTeX notation. The chart also shows that 31% of records use display-math block equations, which is important because the fine-tuned model is expected to generate mathematical formatting.

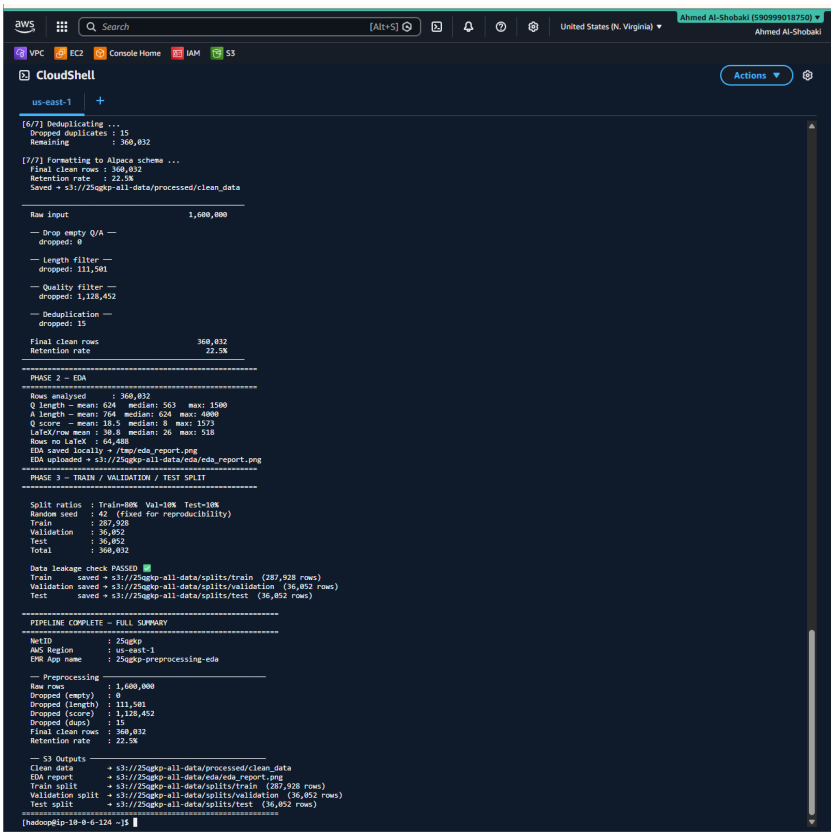

Figure 13: CloudShell output showing the preprocessing summary, final row count, and split verification.

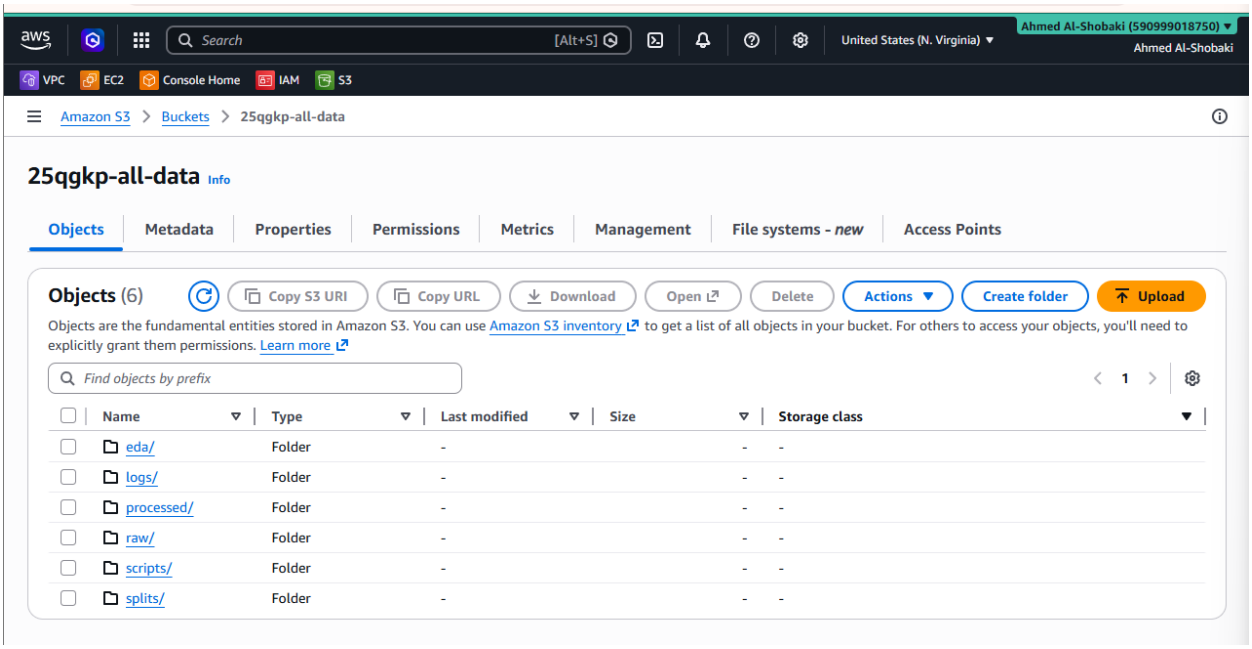

Figure 14: S3 output folders created by the preprocessing pipeline, including processed data, EDA, splits, scripts, and logs.

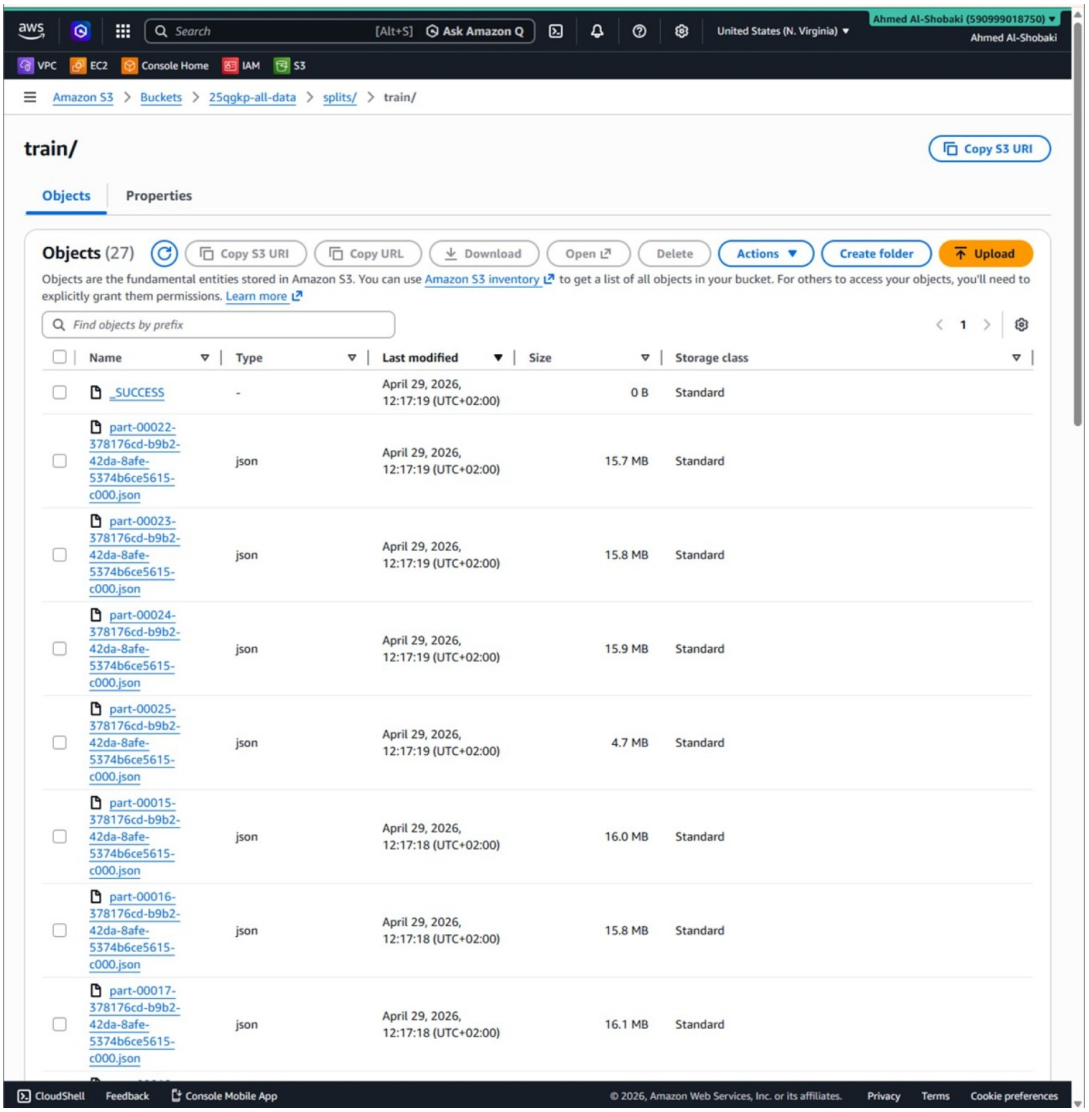

Figure 15: S3 `splits/train/` folder showing PySpark-generated `part-*.json` output files for the training split.

The `splits/train/` folder contains 27 Spark part files ranging from 4.7 MB to 16.1 MB, written on April 29, 2026. The `_SUCCESS` marker file confirms that the Spark write job completed without error. This screenshot is included because the deliverable requires evidence of the preprocessed output files saved to S3, not only the top-level folder structure.

4.7 Teardown Evidence

The EMR cluster was terminated after preprocessing to avoid unnecessary cost in the shared AWS account. This screenshot is important because the project specifically required proof that EMR was not left running.

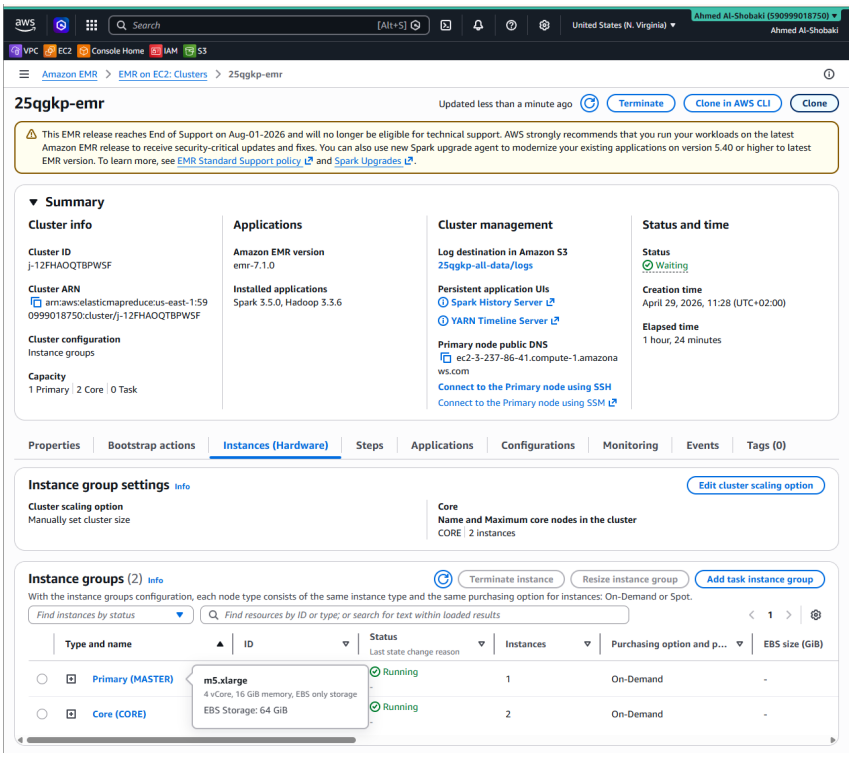

Figure 16: EMR cluster shown in terminated state after preprocessing completed.

5 Model Fine-Tuning

5.1 Fine-Tuning Objective

The fine-tuning objective was to adapt LLaMA 3 8B to answer mathematical questions in the style of StackMathQA: structured, explanatory, and comfortable with LaTeX notation. The goal was not to teach the model all of mathematics from scratch. LLaMA 3 already has general mathematical and language knowledge. Fine-tuning was used to adapt the response style and domain behavior so that the model produces answers that look more like high-quality mathematical explanations.

5.2 Why LoRA and QLoRA Were Used

Full fine-tuning would require updating all 8 billion parameters, which is too expensive for a single L4 GPU and unnecessary for the project goal. LoRA injects small trainable matrices into selected transformer layers while keeping the original model weights frozen. QLoRA combines this with 4-bit quantization, reducing VRAM usage further. This made training feasible on the available hardware while still allowing the model to learn the desired formatting and explanation style.

The selected LoRA target modules were `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. This targets both attention projections and MLP projections, giving the adapters enough capacity to affect token mixing and token-wise transformations.

5.3 Training Environment and Libraries

Component	Specification
Platform	Lightning AI Studios cloud GPU environment; Colab was considered as an alternative
GPU	NVIDIA L4, single device
GPU VRAM	Approximately 20 GB
Precision	4-bit NF4 quantization with automatic bfloat16/float16 detection
Training time	Approximately 76.7 minutes for 600 steps

Library	Version	Role
Unsloth	latest	Faster LoRA training and optimized kernels
Transformers	4.56.2	Model loading, tokenizer, and generation
TRL	0.22.2	`SFTTrainer` supervised fine-tuning loop
PEFT	latest	LoRA adapter injection and management
BitsAndBytes	latest	4-bit NF4 quantization for QLoRA
Datasets	4.3.0	Dataset loading and preprocessing
ROUGE-Score	latest	ROUGE-L evaluation metric

5.4 Hyperparameters

Parameter	Value	Reason
LoRA rank	32	Balances adapter capacity and memory usage
LoRA alpha	64	Standard scaling, double the rank
LoRA dropout	0	No regularization used at this dataset scale
Target modules	q, k, v, o, gate, up, down	Covers attention and MLP projections
Learning rate	2e-4	Common LoRA learning rate for SFT
Batch size per device	4	Fits in memory
Gradient accumulation	4	Effective batch size of 16
Warmup steps	200	Stabilizes training and prevents early divergence
Number of epochs	Approximately 0.19 epoch	600 steps were used instead of a full epoch because 50,000 samples / effective batch 16 is about 3,125 steps per epoch; 600 steps is about 0.19 epoch.
Max steps	600	Chosen due to time/resource constraints

Optimizer	adamw_8bit	Reduces optimizer memory
Scheduler	cosine	Smooth decay during training
Max sequence length	512	Balances context coverage and VRAM
Quantization	4-bit QLoRA	Reduces VRAM for frozen weights
Sequence packing	Enabled	Improves efficiency for shorter samples
Trainable parameters	83,886,080	About 1.03% of the 8B model

5.5 Training Loss

The training loss started at approximately 1.8 and decreased to around 1.3 by step 300. After step 300, the curve mostly plateaued with normal small spikes. This behavior suggests that the model learned the main formatting and domain patterns from the sampled training subset within the 600-step run.

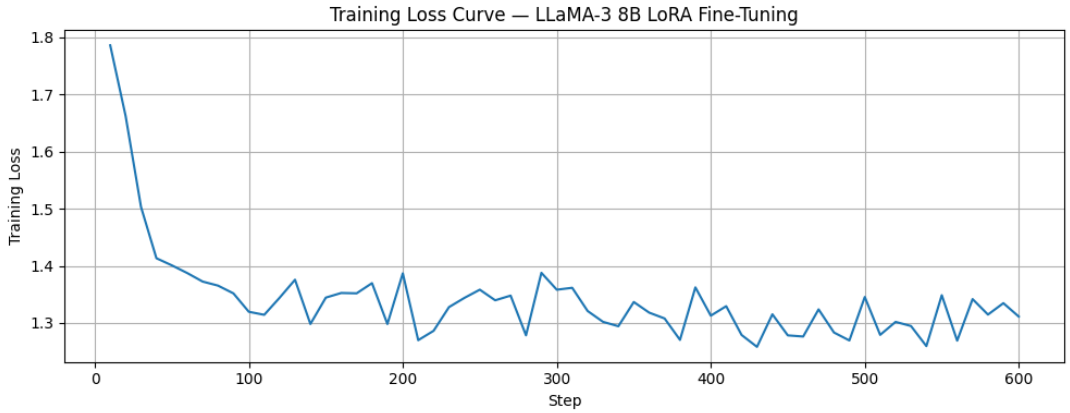

Figure 17: Training loss curve for the LLaMA 3 8B LoRA fine-tuning run.

5.6 Qualitative Comparison: Base vs Fine-Tuned Model

The project deliverable requires at least two example prompts where the base model response is shown alongside the fine-tuned model response. For a fair comparison, the prompt was kept identical for both models in each example. This makes the difference easier to interpret: the base model generally returns a short direct answer, while the fine-tuned model follows the StackMathQA style more closely by adding LaTeX formatting and more explanatory reasoning.

5.6.1 Example 1 - Linear Equation

Base model response

```
### Instruction:
Solve for x: 2x + 5 = 13
### Response:
x = 4
```

Fine-tuned model response

```
### Instruction:\n Solve for x: 2x + 5 = 13\n\n
### Response:
$$2x+5=13$$
$$2x=13-5$$
$$2x=8$$
$$x=8/2$$
$$x=4$$
```

In this example, both models solve the equation correctly. The difference is in presentation. The base model gives only the final value, while the fine-tuned model shows the algebraic manipulation step by step and uses display-math notation. This behavior is closer to the StackMathQA training style, where accepted answers often include intermediate mathematical steps.

5.6.2 Example 2 - Derivative

Base model response

```
### Instruction:
What is the derivative of x^2 + 3x?
### Response:
The derivative of x^2 + 3x is 2x + 3
```

Fine-tuned model response

```
### Instruction:
What is the derivative of x^2 + 3x?
### Response:
Another way to see it is to use the following
    rule: If $f(x)$ is a polynomial function,
    then the derivative of $f(x)$ is also a
    polynomial function whose degree is 1 less
    than the degree of $f(x)$.
Thus, the derivative of $f(x) = x^2 + 3x$ is $f
    '(x) = 2x + 3$. <|end_of_text|>
```

In the second example, the base model again gives a correct short answer. The fine-tuned model adds an explanatory rule before giving the derivative. The output is not perfect because it still includes the model end token in the captured text, but it demonstrates the intended adaptation: more conversational mathematical explanation with LaTeX-style notation.

5.7 Evaluation Results

The model was evaluated on a held-out set of 200 samples using Token F1, ROUGE-L, and a Step Proxy metric. Token F1 and ROUGE-L improved after fine-tuning, indicating better alignment with the reference answers. The Step Proxy decreased slightly, but this was interpreted as a style shift rather than a failure because the fine-tuned model often integrates reasoning inline with LaTeX notation instead of formatting it as numbered steps.

Metric	Base	Fine-tuned	Interpretation
Token F1	0.0877	0.0940	Better vocabulary overlap with references
ROUGE-L	0.1644	0.1744	Better sequence-level similarity
Step Proxy	0.7400	0.6800	Lower due to inline explanation style, not necessarily worse reasoning

5.8 Limitations of Fine-Tuning Evaluation

The evaluation metrics are useful but limited. Reference-based metrics do not fully measure mathematical correctness. A model can use different words or a different proof style and still be mathematically valid. Also, the held-out set had only 200 samples, which is small compared with the diversity of StackMathQA. A stronger future evaluation would use a larger held-out set and an LLM-as-a-judge rubric that checks correctness, clarity, and mathematical reasoning.

6 Model Deployment on EC2

6.1 EC2 Instance Selection

The deployment used an EC2 instance named `25qgkp-ec2` with instance type `t3.xlarge` and Ubuntu 24.04. This instance has 4 vCPUs and 16 GB RAM. It was selected because the final GGUF model was approximately 4.9 GB, and Ollama needs additional memory overhead during inference. A smaller instance could have been tight on RAM, while a larger instance would have increased cost. The final deployment was CPU-only, which is slower than GPU inference but acceptable for a short demo.

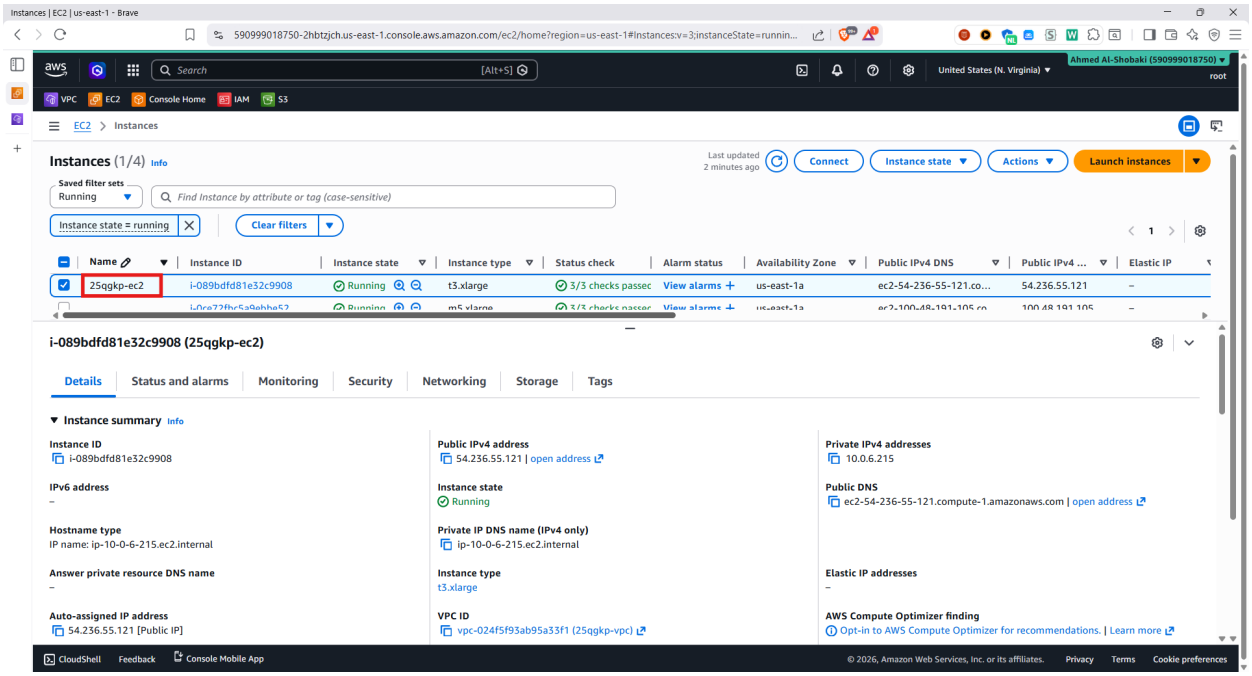

Figure 18: EC2 instance `25qgkp-ec2` running in the public subnet.

6.2 Ollama Installation and Model Loading

Ollama was used as the LLM runner because it supports GGUF models and provides a local HTTP API. The following commands show the deployment flow used on EC2.

```
# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Verify Ollama installation
ollama --version

# Check that the local Ollama API is running
curl http://localhost:11434/api/tags
```

The fine-tuned GGUF model was downloaded from Hugging Face and registered in Ollama using a local `Modelfile`. During deployment testing, we first verified that an Ollama model could be created and listed successfully using the `25qgkp-mathqa` tag. The final browser demo then used the improved `25qgkp-mathqa-merged4:latest` tag, which is visible in the OpenWebUI screenshots. This explains why the terminal model-list evidence and the browser evidence may show different but related tags from the same project deployment workflow.

```
wget -O llama3-math-merged4-q4_k_m.gguf \
  https://huggingface.co/Zainb114/llama3-math-merged4-Q4_K_M-GGUF/resolve/main/llama3-math-merged4-
      q4_k_m.gguf

cat > Modelfile << 'EOF'
FROM ./llama3-math-merged4-q4_k_m.gguf
EOF

ollama create 25qgkp-mathqa-merged4 -f Modelfile
ollama list
```

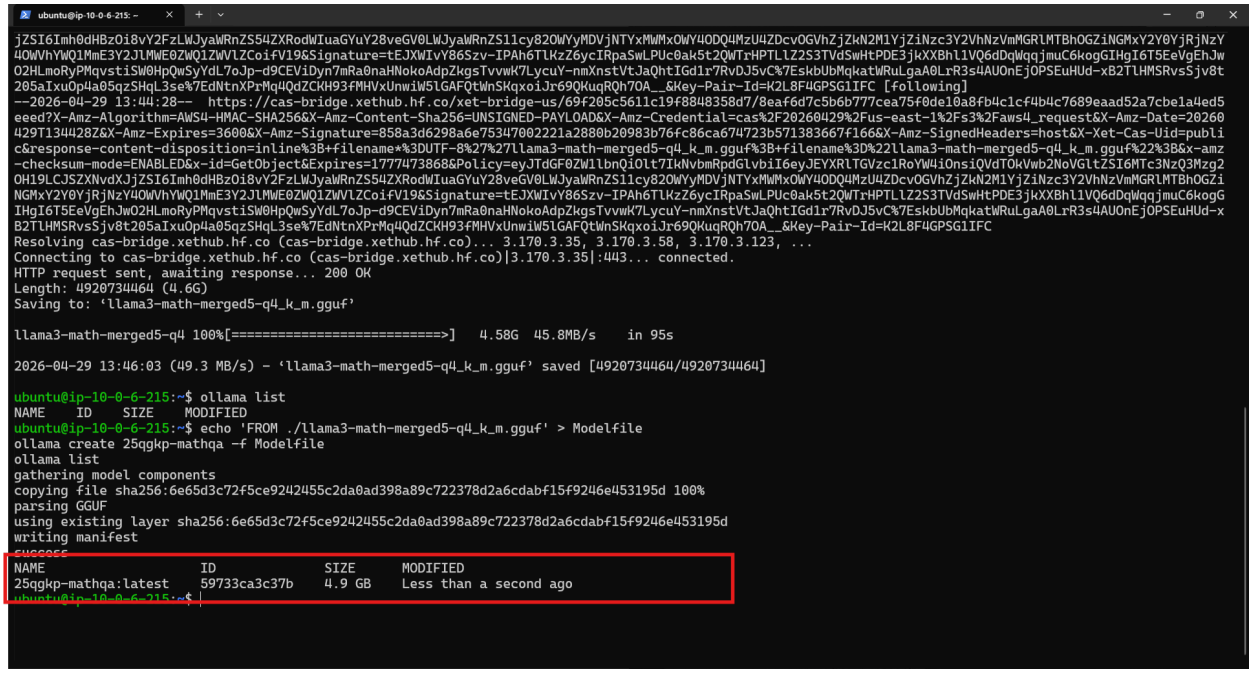

Figure 19: Ollama model list showing that a 4.9 GB GGUF model was successfully registered on EC2. The final browser demo used the merged4 model tag shown later in OpenWebUI.

6.3 API Test with curl

The curl test confirmed that the model could be called through Ollama's local API. The prompt was formatted in a simple question/response style to match the fine-tuning data style.

```
curl -s http://localhost:11434/api/generate -d '{
  "model": "25qgkp-mathqa-merged4",
  "prompt": "### Question:\nWhat is the derivative of x^2? Answer in one sentence.\n\n### Response:\n
      ",
  "stream": false,
  "options": {
    "num_predict": 40,
    "temperature": 0,
    "stop": ["This can be shown", "Let "]
  }
}' | python3 -c 'import sys,json; d=json.load(sys.stdin); print("Question: What is the derivative of x
    ^2?\n"); print("Response:"); print(d["response"].strip())'
```

The output was:

```
Question: What is the derivative of x^2?

Response:
The derivative of $x^2$ is $2x$.
```

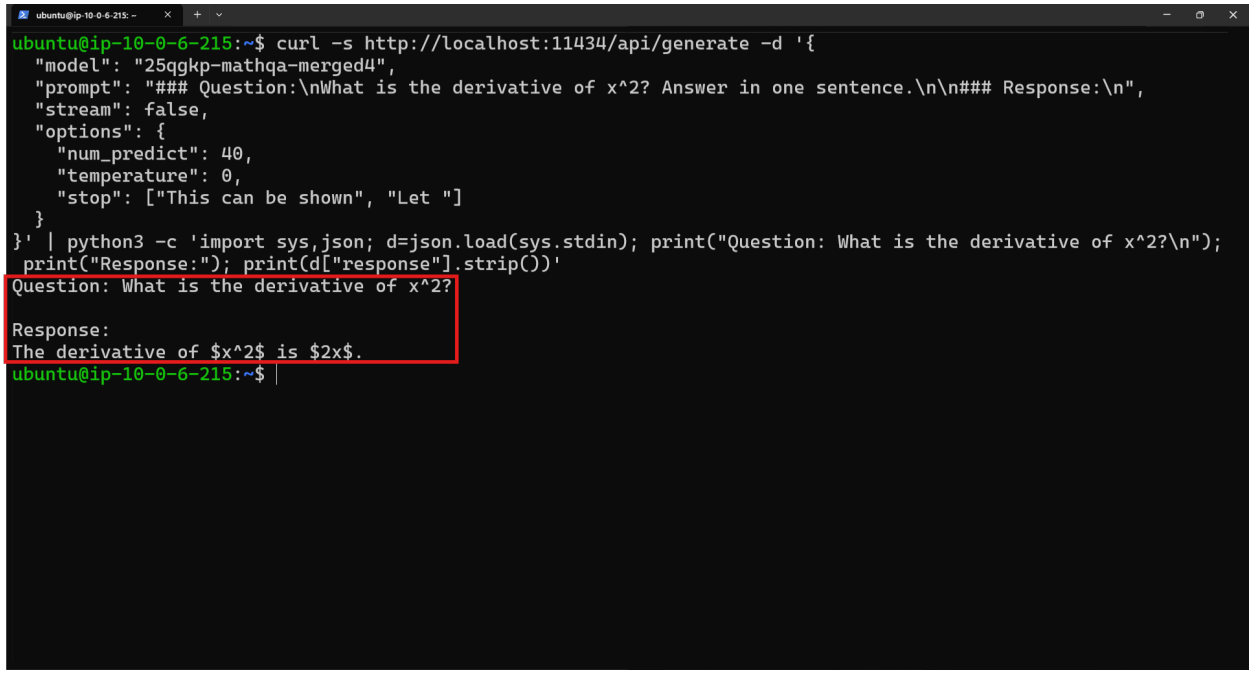

Figure 20: curl test showing a response from the fine-tuned model through Ollama.

7 Web Interface with OpenWebUI

7.1 Why OpenWebUI Was Used

OpenWebUI was used to provide a browser-based interface for the model. This is important because the project required a live web interface, not only terminal API testing. OpenWebUI connects to Ollama and allows the user to select the fine-tuned model from the browser.

7.2 Docker Deployment Command

Docker was installed on EC2, and OpenWebUI was launched as a container. The container used host networking so it could reach Ollama at `127.0.0.1:11434`. The restart policy `-restart always` was included so the interface would restart automatically if the server restarted.

```
sudo docker run -d \
  --name open-webui \
  --network host \
  -v open-webui:/app/backend/data \
  -e OLLAMA_BASE_URL=http://127.0.0.1:11434 \
  --restart always \
  ghcr.io/open-webui/open-webui:main

sudo docker ps
```

7.3 OpenWebUI Evidence

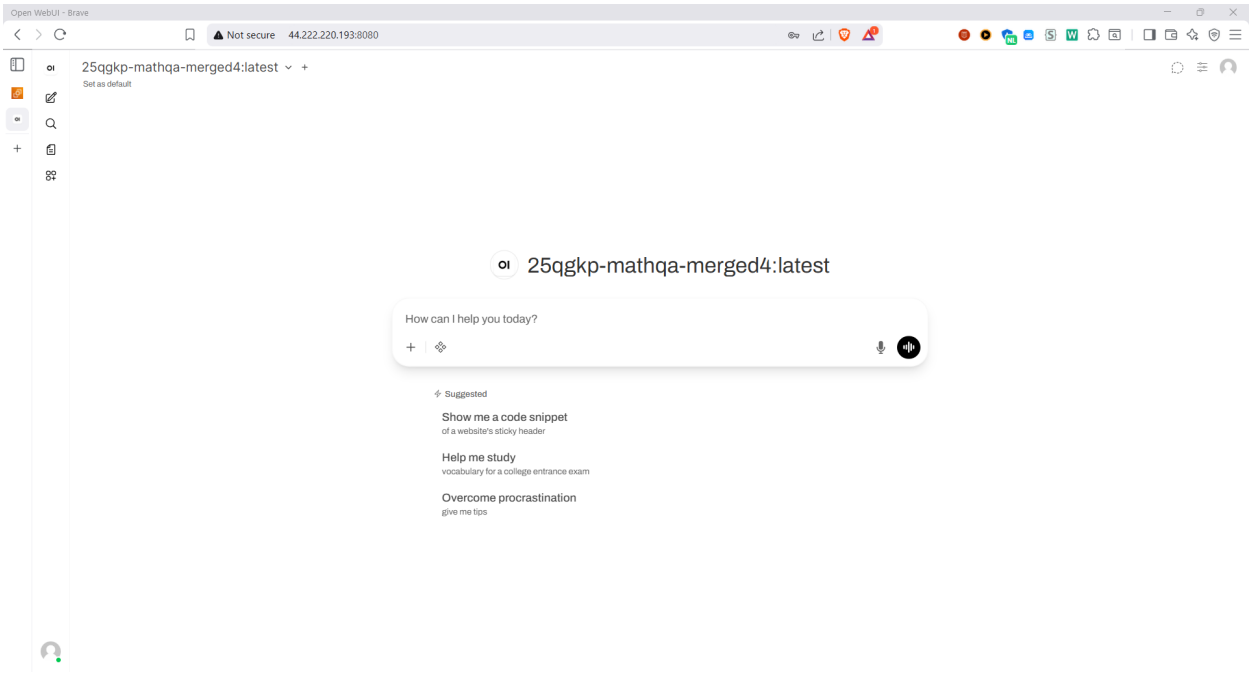

Figure 21: OpenWebUI running in the browser with the fine-tuned model name visible.

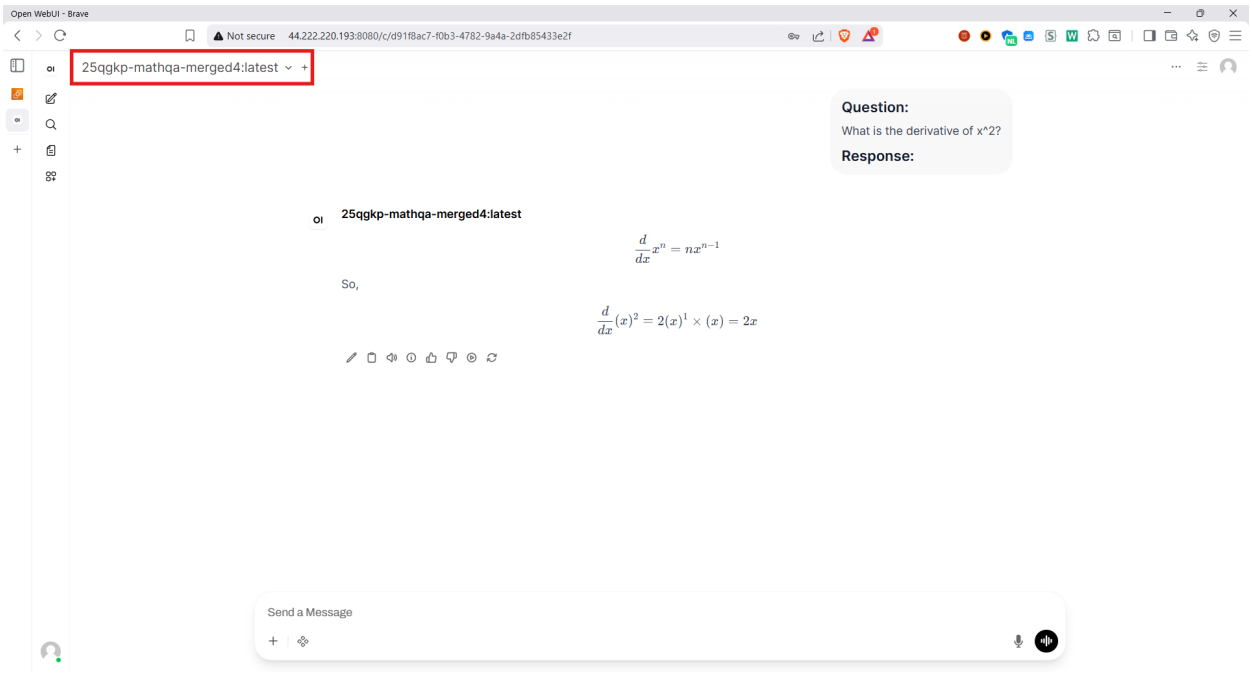

Figure 22: Sample browser chat using the deployed fine-tuned math model.

8 Cost Summary, Free Tier/Credits, and Cleanup

8.1 How Cost Was Checked

Two AWS billing views were used. First, Cost Explorer was filtered to the active project window, grouped by service, and limited to `US East (N. Virginia)`. Second, the Amazon Q billing summary, shown from the same AWS Billing and Cost Management area, was used to interpret the exact daily

usage charges and credits applied for April 28–29, 2026. Cost Explorer is kept as the service-level AWS billing evidence, while the Amazon Q summary explains why the net paid amount appears as zero.

8.2 Actual Usage, Credits, and Net Paid

The important distinction is that the project did create billable usage, but the account credits covered that usage. Therefore, the net amount paid was zero, but the project should not be described as if EMR or EC2 were inherently free.

Date	Usage charges	Credits applied	Net paid
April 28, 2026	$0.27	-$0.27	$0.00
April 29, 2026	$2.50	-$2.50	$0.00
Total	**$2.77**	**-$2.77**	**$0.00**

8.3 Cost Explorer Service View

Cost Explorer was also grouped by service for the selected project period. In that view, each visible service total appeared as approximately $0.00 after credits were applied. This table therefore reports the accurate *visible net Cost Explorer amount* by service rather than inventing a pre-credit service allocation that was not available from the screenshot.

Service shown in Cost Explorer	Project purpose	Visible net amount
EC2-Instances	EC2 deployment host for Ollama/OpenWebUI	$0.00
EC2-Other	EC2-related support costs such as EBS/data path items	$0.00
Elastic MapReduce	EMR service used for PySpark preprocessing	$0.00
S3	Raw data, processed outputs, scripts, logs, and figures	$0.00
VPC	Networking support for the deployment	$0.00
CloudWatch	Logs/metrics generated by AWS services	$0.00
Glue	Supporting catalog/service visibility in the billing view	$0.00
Data Transfer	Small network transfer adjustments	-$0.00
Total visible estimate	Project period after credits	**$0.00**

8.4 Free-Tier and Normally Billable Components

From the architecture, some resources have no direct hourly cost, some may fall inside free-tier or credit coverage, and some are normally billable.

Architecture Component	Cost Category	Explanation
VPC, subnet, route table, security group	No direct hourly charge	These networking constructs generally do not create compute cost by themselves.

Internet Gateway	No direct hourly charge	The gateway itself has no hourly charge, but traffic/data transfer can still matter.
EC2 `t3.xlarge`	Normally billable	Not a micro free-tier instance. It was used temporarily and then terminated.
EBS root volume	Potentially billable/free-tier-covered	Storage can be covered by free-tier/credits depending on account eligibility and size.
S3 bucket and objects	Potentially billable/free-tier-covered	Used for raw data, processed outputs, scripts, logs, and figures.
EMR `m5.xlarge` cluster	Normally billable	EMR service and underlying EC2 instances are not treated as always-free resources.
Ollama, Docker, OpenWebUI	No AWS software charge	Open-source software running on EC2; compute still comes from EC2.

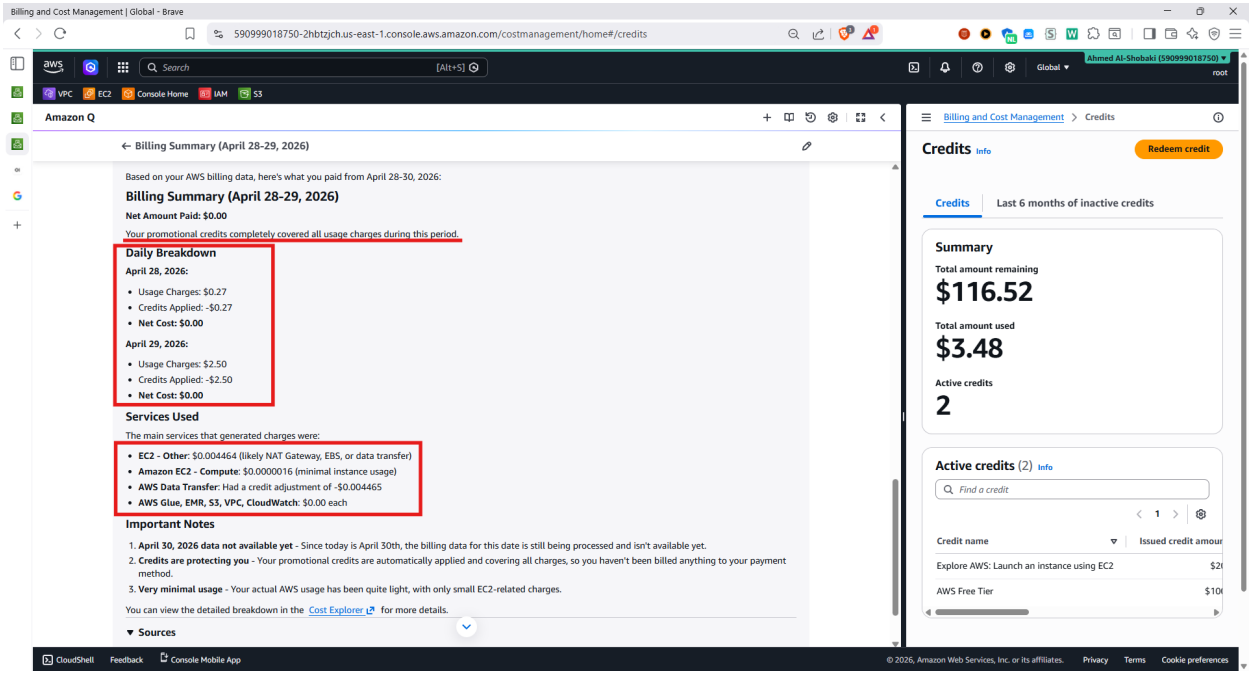

Figure 23: Amazon Q billing summary showing daily usage charges, credits applied, and net paid amount.

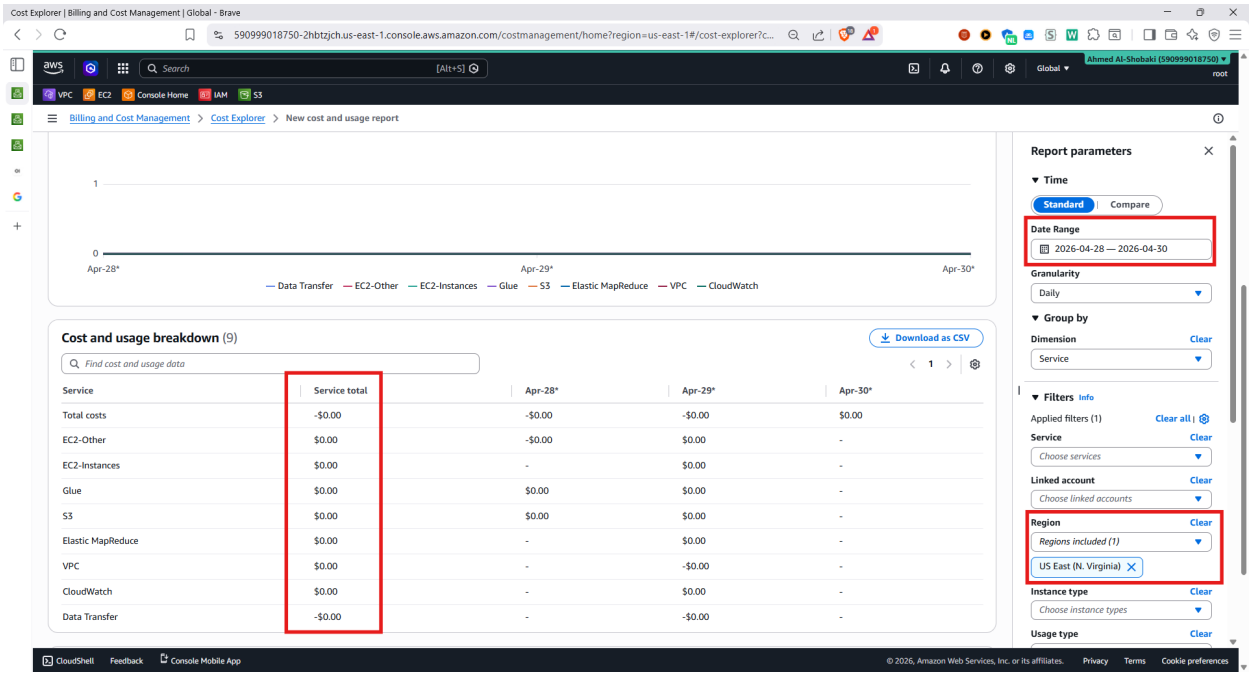

Figure 24: Cost Explorer filtered by project period, service, and US East (N. Virginia). The effective service totals appear as zero because credits offset the usage charges and current-period estimates can lag.

8.5 Cleanup

All temporary compute resources were terminated after screenshots and testing. This was important because EMR and EC2 are normally billable services if left running.

Resource	Final status
EMR cluster `25qgkp-emr`	Terminated
EC2 instance `25qgkp-ec2`	Terminated after deployment screenshots were captured
OpenWebUI container	Removed with EC2 termination
Ollama deployment files	Removed with EC2 termination
S3 project bucket	Kept only as needed for final artifacts / can be removed after grading

9 GitHub Repository and Reproducibility

The GitHub repository contains the code and instructions needed to reproduce the main phases of the project. The README includes prerequisites, AWS region, naming policy, S3/EMR commands, fine-tuning steps, EC2/Ollama/OpenWebUI deployment commands, cost summary, and cleanup instructions. This is necessary because the project requirements state that a working README is required for replication.

```
Cloud_project_Group6/
  README.md
  data_preprocessing/
    25qgkp_emr_preprocessing.py
    bootstrap.sh
  fine_tuning/
    Llama3_8B_final_finetuning_v3.ipynb
  deployment/
    ec2_ollama_openwebui_commands.md
```

```
figures/
  eda_report.png
  training_loss_curve.png
screenshots/
  section1-2-infrastructure/
  section4-emr-spark/
  section6-7-deployment/
```

10 Conclusion

The final project satisfies the full end-to-end cloud assistant workflow. It uses a custom VPC rather than the default VPC, stores data in S3, preprocesses the dataset with EMR and PySpark, fine-tunes LLaMA 3 8B with QLoRA, exports a GGUF model artifact, serves the model with Ollama on EC2, and exposes the assistant through OpenWebUI. The final system was tested through both curl and browser interaction.

The main engineering tradeoff was choosing a CPU-only EC2 deployment instead of GPU inference. This reduced cost and resource pressure in the shared account, but it also made inference slower. For a production system, the next step would be to use a GPU-backed serving endpoint, restrict security group sources, add HTTPS, automate infrastructure with Terraform, and use a stronger evaluation method for mathematical correctness.

A Appendix A: Commands and Code Snippets Used

This appendix collects the main commands used in the project. Values such as `<cluster-id>` and `<EC2_PUBLIC_IP>` should be replaced with the actual values from the AWS Console. The EC2 public IP is intentionally written as a placeholder because it can change if the instance is stopped and restarted without an Elastic IP.

A.1 S3 Bucket and Raw Dataset Upload

```
# Create project S3 bucket in us-east-1
aws s3 mb s3://25qgkp-all-data --region us-east-1

# Upload the raw StackMathQA JSONL file
aws s3 cp all.jsonl s3://25qgkp-all-data/raw/all.jsonl

# Upload the preprocessing script and bootstrap script
aws s3 cp 25qgkp_emr_preprocessing.py \
  s3://25qgkp-all-data/scripts/25qgkp_emr_preprocessing.py
aws s3 cp bootstrap.sh \
  s3://25qgkp-all-data/scripts/bootstrap.sh
```

A.2 EMR Bootstrap Script

```
cat > /tmp/bootstrap.sh << 'EOF'
#!/bin/bash
sudo pip3 install --upgrade pip
sudo pip3 install matplotlib numpy pandas pyarrow fsspec s3fs boto3
sudo pip3 install datasets --ignore-installed --no-deps
sudo pip3 install huggingface-hub tqdm requests filelock --ignore-installed
EOF

aws s3 cp /tmp/bootstrap.sh s3://25qgkp-all-data/scripts/bootstrap.sh
```

A.3 EMR Cluster Creation

```
aws emr create-cluster \
  --name "25qgkp-emr" \
  --release-label emr-7.1.0 \
  --applications Name=Spark \
  --instance-groups \
    InstanceGroupType=MASTER,InstanceCount=1,InstanceType=m5.xlarge \
    InstanceGroupType=CORE,InstanceCount=2,InstanceType=m5.xlarge \
  --bootstrap-actions Name="Install Python libs",Path=s3://25qgkp-all-data/scripts/bootstrap.sh \
  --log-uri s3://25qgkp-all-data/logs/ \
  --region us-east-1 \
  --ec2-attributes KeyName=25qgkp-keypair \
  --use-default-roles
```

A.4 Run the PySpark Preprocessing Job

```
# SSH to the EMR master node after replacing <cluster-id>
ssh -i ~/25qgkp-keypair.pem hadoop@$(aws emr describe-cluster \
  --cluster-id <cluster-id> \
  --region us-east-1 \
  --query "Cluster.MasterPublicDnsName" \
  --output text)
```

```
# Submit the preprocessing job
spark-submit \
  --master yarn \
  --deploy-mode client \
  s3://25qgkp-all-data/scripts/25qgkp_emr_preprocessing.py
```

A.5 Fine-Tuning Configuration Snippet

```
model_name = "unsloth/llama-3-8b-bnb-4bit"
max_seq_length = 2048
load_in_4bit = True

lora_r = 16
lora_alpha = 16
lora_dropout = 0

training_args = {
    "per_device_train_batch_size": 2,
    "gradient_accumulation_steps": 4,
    "warmup_steps": 200,
    "max_steps": 600,
    "learning_rate": 2e-4,
    "fp16": True,
    "optim": "adamw_8bit",
    "weight_decay": 0.01,
    "lr_scheduler_type": "cosine",
    "seed": 3407,
}
```

A.6 SSH into EC2 and Install Ollama

```
# Connect to the EC2 deployment instance
ssh -i "25qgkp-keypair.pem" ubuntu@<EC2_PUBLIC_IP>

# Install and verify Ollama
curl -fsSL https://ollama.com/install.sh | sh
ollama --version
curl http://localhost:11434/api/tags
```

A.7 Download GGUF and Register the Ollama Model

```
wget -O llama3-math-merged4-q4_k_m.gguf \
  https://huggingface.co/Zainb114/llama3-math-merged4-Q4_K_M-GGUF/resolve/main/llama3-math-merged4-
      q4_k_m.gguf

cat > Modelfile << 'EOF'
FROM ./llama3-math-merged4-q4_k_m.gguf
EOF

ollama create 25qgkp-mathqa-merged4 -f Modelfile
ollama list
```

A.8 Test the Model through the Ollama API

```
curl -s http://localhost:11434/api/generate -d '{
  "model": "25qgkp-mathqa-merged4",
  "prompt": "### Question:\nWhat is the derivative of x^2? Answer in one sentence.\n\n### Response:\n
      ",
```

```
  "stream": false,
  "options": {
    "num_predict": 40,
    "temperature": 0,
    "stop": ["This can be shown", "Let "]
  }
}' | python3 -c 'import sys,json; d=json.load(sys.stdin); print("Question: What is the derivative of x
    ^2?\n"); print("Response:"); print(d["response"].strip())'
```

A.9 Install Docker and Run OpenWebUI

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable docker
sudo systemctl start docker

sudo docker run -d \
  --name open-webui \
  --network host \
  -v open-webui:/app/backend/data \
  -e OLLAMA_BASE_URL=http://127.0.0.1:11434 \
  --restart always \
  ghcr.io/open-webui/open-webui:main

sudo docker ps
curl http://localhost:8080
```

A.10 Cleanup Commands

```
# Terminate EMR after preprocessing
aws emr terminate-clusters --cluster-ids <cluster-id> --region us-east-1

# EC2 was terminated from the AWS Console after screenshots and testing.
# S3 artifacts were kept only as long as they were needed for submission evidence.
```